

Sistemas operativos

Ejecución de procesos

Gestión de lo procesos: Multiprogramación o Multitarea

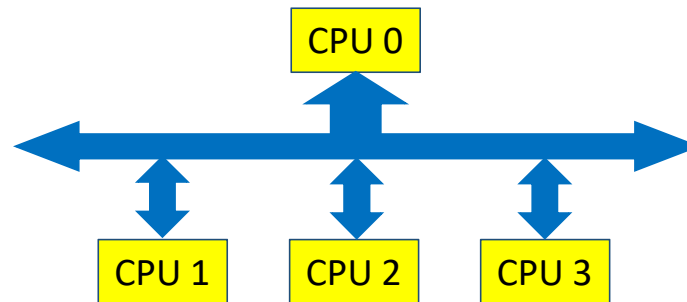
Multiprogramación o Multitarea

- Se creó para paliar la baja utilización de los recursos por parte de la monoprogramación.
- Es una técnica de multiplexación (**conmutación**) que permite la ***ejecución simultánea*** de múltiples procesos en un único procesador. En realidad, esta técnica produce una ilusión de paralelismo, de manera que parece que todos los procesos se están ejecutando a la vez. Sin embargo, hay un único proceso ejecutándose en el procesador a la vez durante un período de tiempo determinado.
- Incluso en computadoras **multiprocesador** (verdadero paralelismo), la multitarea permite ejecutar muchas más tareas en cada procesador que conforma la máquina.
- Se apoya en varios **elementos del hardware**, como interrupción, DMA, canal.
- Todos los recursos del computador (CPU, Memoria, Periféricos) son **compartidos** por los diferentes procesos.
- También se encuentran todos los sistemas que cumplen simultáneamente las necesidades de dos o más usuarios (**sistemas multiusuario**) que compartan los mismos recursos. Este tipo de sistemas se emplea especialmente en redes.
- El procesamiento que se da en este tipo de Sistemas Operativos incluyen mecanismos detallados (**algoritmos de planificación**) para **cambiar el contexto** (proceso en ejecución) para gestionar los procesos y los recursos de la máquina huésped.

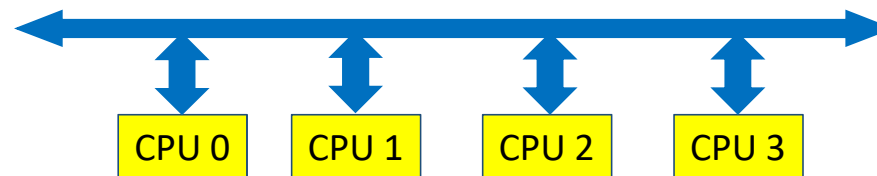
Sistema de multiprocesamiento

- Son sistemas con 2 o más núcleos (procesadores) trabajando en paralelo, compartiendo la memoria principal y otros recursos.
- Cada procesador puede ejecutar un diferente hilo.

Multiprocesamiento asimétrico: Un procesador controla al resto (algunas versiones de UNIX).

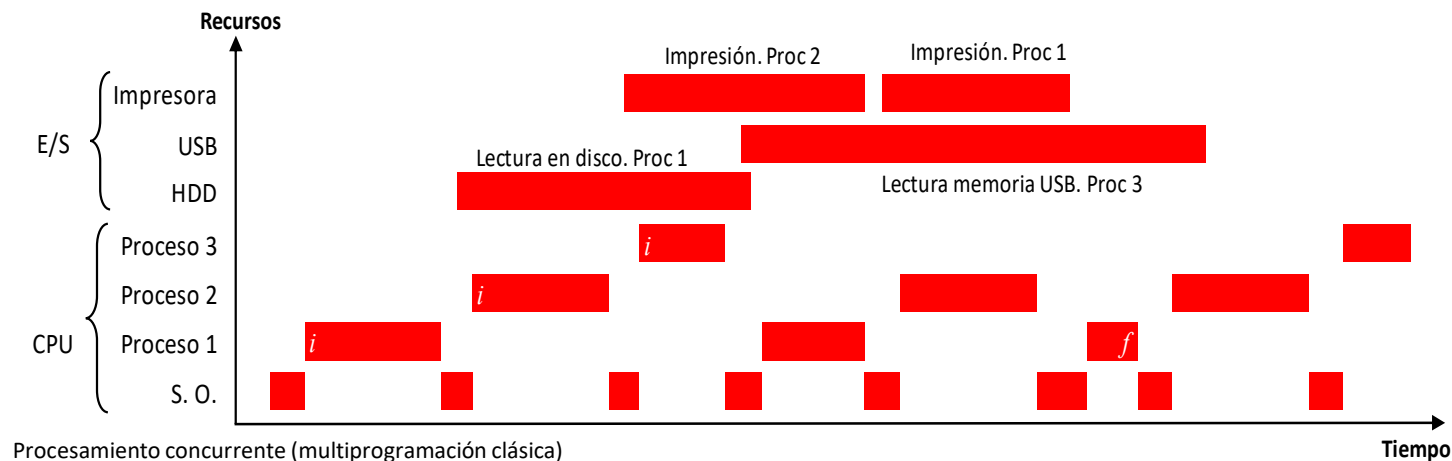
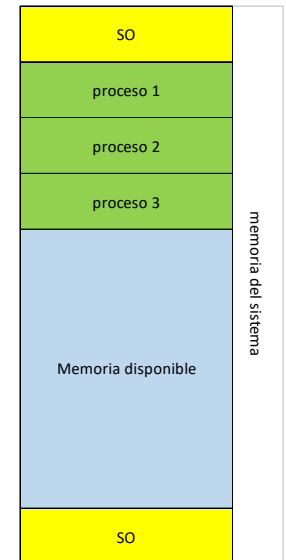


Multiprocesamiento simétrico: Todos los procesadores son iguales, incrementando linealmente las prestaciones (por ejemplo, el núcleo de Windows NT)

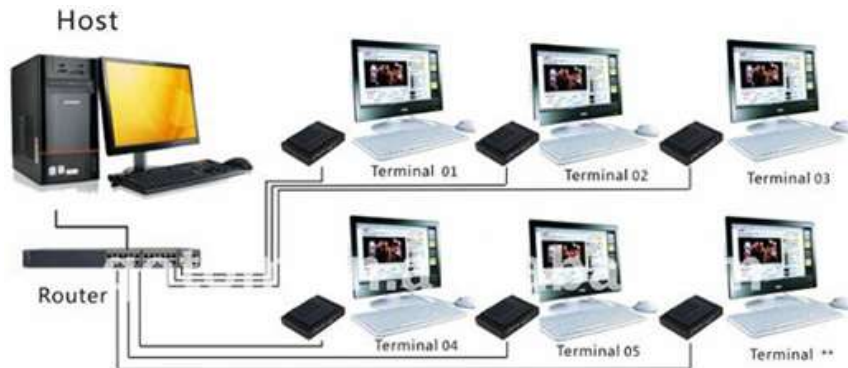


Multiprogramación o Multitarea

- En un sistema multiprogramado, la memoria principal alberga a más de un proceso de usuario. Un proceso es un programa cargado en memoria.
- La CPU ejecuta instrucciones de un proceso en ejecución (el que ocupa la CPU).
- Si el proceso en ejecución realiza una operación de E/S, emite ciertos órdenes al controlador (igual que en los sistemas monoprogramados); pero en lugar de esperar a que termine la operación de E/S comprobando el bit de ocupación, se pasa a ejecutar otro proceso. Esto permite que varios dispositivos trabajen simultáneamente, además, en la CPU no se tienen que ejecutar ciclos de comprobación del estado de los dispositivos.
- Cuando el SO asigna el procesador a los diferentes procesos, el mecanismo se conoce como **planificador a corto plazo**. Así van avanzando en su ejecución.



- Después de la aparición de la multiprogramación surgieron los ordenadores de acceso múltiple o **multiusuario**. En ellos cada usuario dispone de un **terminal**, es decir, un teclado y una pantalla conectados al ordenador.
- Los usuarios ejecutan programas interactivos en estas terminales.
- Un programa interactivo es aquel que se comunica con el usuario por medio de un terminal, el usuario le suministra información al programa mediante el teclado, y recibe información del programa a través de la pantalla.
- Los programas de los usuarios comparten los recursos (CPU, memoria, discos, impresoras, etc) del ordenador.
- Estos sistemas hacen uso de una variante de la multiprogramación llamada **tiempo compartido**.



En el tiempo compartido los programas de los distintos usuarios residen en memoria; al realizar una operación de E/S los programas ceden la CPU a otro programa, al igual que en la multiprogramación. Pero, a diferencia de ésta, cuando un programa lleva cierto tiempo ejecutándose sin realizar operaciones de E/S el sistema operativo lo detiene para que se ejecute otro programa. Con esto se consigue repartir la CPU con equidad entre los programas de los distintos usuarios. Al tener la CPU una capacidad de cálculo tan grande, los programas de los usuarios no se sienten demasiado ralentizados por el hecho de que la CPU (y los demás recursos) sean compartidos.

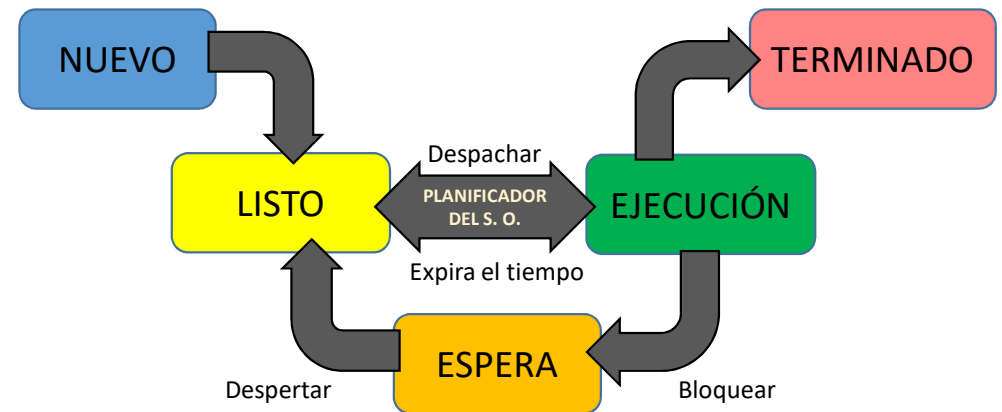
Control de procesos

Estados de proceso

Los procesos tienen un estado de ejecución en un momento dado, esto es, los valores de los registros de la CPU para dicho programa, la memoria que ha reservado y sus contenidos, así como otra información que permite al sistema operativo su planificación. Esta definición varía ligeramente en el caso de sistemas operativos multihilo, donde un proceso consta de uno o más hilos, la memoria de trabajo (compartida por todos los hilos) y la información de planificación. Cada hilo consta de instrucciones y estado de ejecución.

En los sistemas operativos multihilo es posible crear tanto hilos como procesos. La diferencia está en que un proceso solamente puede crear hilos para sí mismo y en que dichos hilos comparten toda la memoria reservada para el proceso.

- **Nuevo y terminado:** Los procesos son creados y destruidos por el sistema operativo, así como también este se debe hacer cargo de la **comunicación entre procesos**, pero lo hace a petición de otros procesos. El mecanismo por el cual un proceso crea otro proceso se denomina bifurcación (fork). Los nuevos procesos son independientes y no comparten memoria (es decir, información) con el proceso que los ha creado.



Estados de proceso

- **En ejecución:** El proceso ocupa la CPU actualmente, es decir, se está ejecutando.
- **Listo o preparado:** El proceso dispone de todos los recursos para su ejecución, sólo le falta la CPU.
- **Espera o Bloqueado:** Al proceso le falta algún recurso para poder seguir ejecutándose, además de la CPU. Por recurso se pueden entender un dispositivo, un dato, etc. El proceso necesita que ocurra algún evento que le permita poder proseguir su ejecución.

Hay otros estados de los procesos, pero en principio se tratarán estos tres. Por sencillez, se considera un sistema con una sola CPU, aunque no es difícil la extensión a múltiples procesadores. Solamente puede haber un proceso en ejecución a la vez, pero pueden existir varios listos y varios pueden estar bloqueados. Así pues, se forman una lista de procesos listos y otra de procesos bloqueados. La lista de procesos listos se ordena por prioridad, de manera que el siguiente proceso que reciba la CPU será el primero de la lista. La lista de procesos bloqueados normalmente no está ordenada; los procesos no se desbloquean (es decir, no pasan a ser procesos listos) en orden de prioridad, sino que lo hacen en el orden de ocurrencia de los eventos que están esperando. Como se verá más adelante, hay situaciones en las cuales varios procesos pueden bloquearse esperando la ocurrencia del mismo evento; en tales casos es común asignar prioridades a los procesos que esperan.

Transiciones de estado de los procesos

A continuación se dan ejemplos de eventos que pueden provocar transiciones de estado en un proceso en este modelo de tres estados:

De ejecución a Bloqueado: al iniciar una operación de E/S, al realizar una operación WAIT sobre un semáforo a cero (en el tema de procesos concurrentes se estudiarán los semáforos).

De ejecución a Listo: por ejemplo, en un sistema de tiempo compartido, cuando el proceso que ocupa la CPU lleva demasiado tiempo ejecutándose continuamente (agota su cuanto) el sistema operativo decide que otro proceso ocupe la CPU, pasando el proceso que ocupaba la CPU a estado listo.

- De Listo a en ejecución: cuando lo requiere el planificador de la CPU (veremos el planificador de la CPU en el tema de planificación de procesos).
- De Bloqueado a Listo: se dispone del recurso por el que se había bloqueado el proceso. Por ejemplo, termina la operación de E/S, o se produce una operación SIGNAL sobre el semáforo en que se bloqueó el proceso, no habiendo otros procesos bloqueados en el semáforo.

Obsérvese que de las cuatro transiciones de estado posibles, la única iniciada por el proceso de usuario es el bloqueo, las otras tres son iniciadas por entidades externas al proceso.

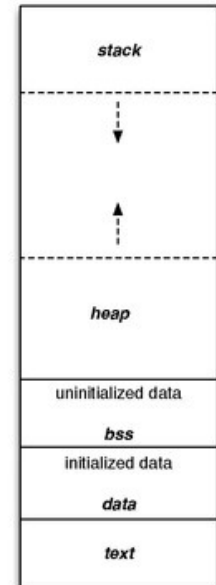
Descripción de un proceso

¿cuál es la manifestación física de un proceso?

- Como mínimo debe incluir un conjunto de instrucciones para que sean ejecutados.
- Asociados a estos programas hay un conjunto de ubicaciones de datos para las variables locales y globales, y las constantes definidas.

Un proceso constará, al menos, de la memoria suficiente para albergar los programas y los datos del proceso.

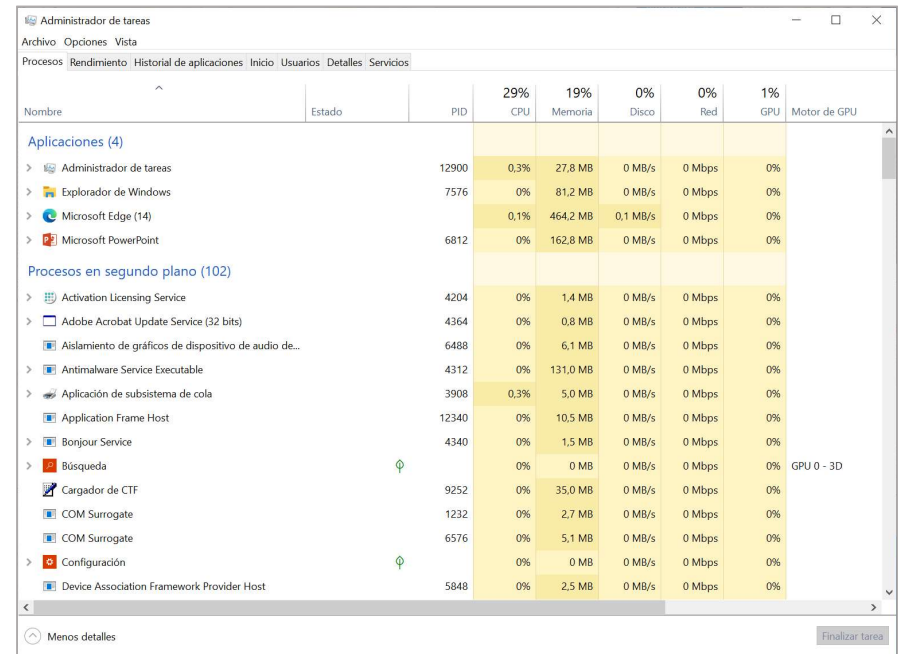
- Además, en la ejecución de un programa entra en juego normalmente una pila, que se utiliza para llevar la cuenta de las llamadas a procedimientos y de los parámetros que se pasan entre los procedimientos.
- Asociado a cada proceso hay una serie de atributos que utiliza el sistema operativo para el control del proceso. Estos atributos se recogen en una estructura de datos que se conoce como **bloque de control de proceso** (Process Control Block, PCB) o descriptor de proceso.
- A esta colección de programa, datos, pila y atributos se le llama imagen o entorno del proceso.



Bloque de control de procesos (PCB)

Es la estructura de datos central y más importante del sistema operativo. Se usa para almacenar toda la información sobre un proceso para su control. También se conoce como descriptor de proceso. El conjunto de todos los PCB's se guarda en una estructura del SO llamada **tabla de procesos** que contiene los registros de estas unidades. Estas existen cuando:

- Se crea un proceso (inicializado o instalado), el sistema operativo crea un bloque de control de proceso correspondiente.
- La información de un bloque de control de procesos se actualiza durante la transición de los estados de proceso.
- Cuando el proceso termina, su PCB se devuelve al pool del cual se dibujan los nuevos PCB.
- Cada proceso tiene una sola PCB.
- El PCB es la entidad que define un proceso en el sistema operativo. Dado que los PCB necesitan ser manejados con eficiencia por el sistema operativo, muchos ordenadores tienen un registro hardware que siempre apunta hacia el PCB del proceso que se está ejecutando. A menudo existen instrucciones hardware que cargan en el PCB información sobre su entorno, y la recuperan con rapidez.



Nombre	Estado	PID	29% CPU	19% Memoria	0% Disco	0% Red	1% GPU	Motor de GPU
Aplicaciones (4)								
Administrador de tareas		12900	0.3%	27.8 MB	0 MB/s	0 Mbps	0%	
Explorador de Windows		7576	0%	81.2 MB	0 MB/s	0 Mbps	0%	
Microsoft Edge (14)			0.1%	464.2 MB	0.1 MB/s	0 Mbps	0%	
Microsoft PowerPoint		6812	0%	162.8 MB	0 MB/s	0 Mbps	0%	
Procesos en segundo plano (102)								
Activation Licensing Service		4204	0%	1.4 MB	0 MB/s	0 Mbps	0%	
Adobe Acrobat Update Service (32 bits)		4364	0%	0.8 MB	0 MB/s	0 Mbps	0%	
Aislamiento de gráficos de dispositivo de audio de...		6488	0%	6.1 MB	0 MB/s	0 Mbps	0%	
Antimalware Service Executable		4312	0%	131.0 MB	0 MB/s	0 Mbps	0%	
Aplicación de subsistema de cola		3908	0.3%	5.0 MB	0 MB/s	0 Mbps	0%	
Application Frame Host		12340	0%	10.5 MB	0 MB/s	0 Mbps	0%	
Bonjour Service		4340	0%	1.5 MB	0 MB/s	0 Mbps	0%	
Búsqueda			0%	0 MB	0 MB/s	0 Mbps	0%	GPU 0 - 3D
Cargador de CTF		9252	0%	35.0 MB	0 MB/s	0 Mbps	0%	
COM Surrogate		1232	0%	2.7 MB	0 MB/s	0 Mbps	0%	
COM Surrogate		6576	0%	5.1 MB	0 MB/s	0 Mbps	0%	
Configuración			0%	0 MB	0 MB/s	0 Mbps	0%	
Device Association Framework Provider Host		5848	0%	2.5 MB	0 MB/s	0 Mbps	0%	

Bloque de control de procesos (PCB)

En un sistema de multiprogramación, se requiere una gran cantidad de información de cada proceso para su administración. Sistemas distintos organizarán esta información de modo diferente. En general, se puede agrupar la información de los PCB's en tres categorías:

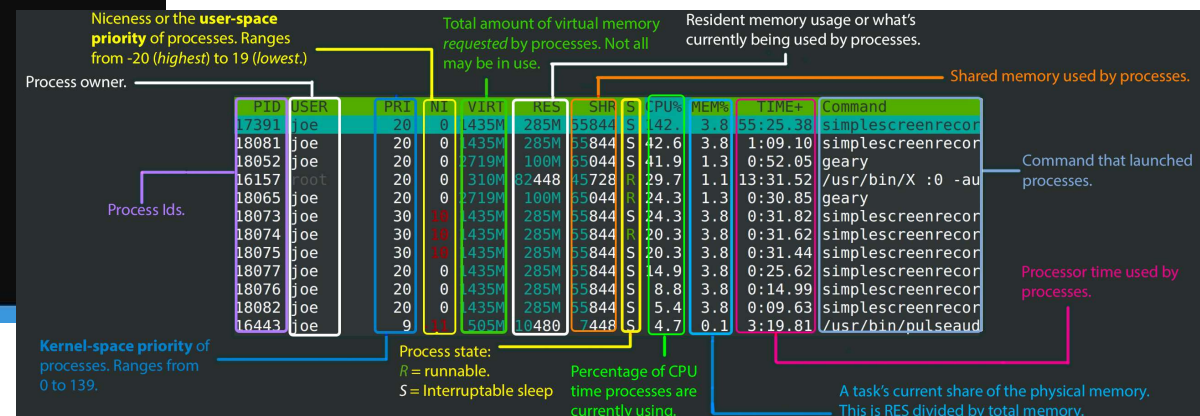
- **Identificación del proceso:** todos los sistemas operativos se le asigna a cada proceso un identificador numérico único (PID). Este identificador nos servirá para localizarlo dentro de la tabla de procesos. Cuando se permite que los procesos creen otros procesos, se utilizan identificadores para señalar al padre y a los descendientes de cada proceso. Además de estos, un proceso también puede tener asignado un identificador de usuario que indica a quién pertenece el proceso (UID).
- **Información del estado del procesador:** Básicamente, está formada por el contenido de los registros del procesador. Por supuesto, mientras el proceso está ejecutándose, la información está en los registros. Cuando se interrumpe el proceso, toda la información de los registros debe salvarse de forma que pueda restaurarse cuando el proceso reanude su ejecución. La naturaleza y número de registros involucrados depende del diseño del procesador. Normalmente, en el conjunto de registros se incluyen los registros visibles para el usuario, los registros de control y de estado (contador de programa y palabra de estado) y los punteros a pila.
- **Información de control del proceso:** Esta es la información adicional necesaria para que el sistema operativo controle y coordine los diferentes procesos activos. Como, por ejemplo, información de planificación y estado (estado del proceso, su prioridad, información de planificación, suceso), apuntadores(punteros) a estructuras de datos (los procesos que esperan en un semáforo), punteros a zonas de memoria del proceso, recursos controlados por el proceso (ficheros abiertos), etc.

Bloque de control de procesos (PCB)

- En Linux, cada proceso del sistema tiene dos estructuras que lo identifican: el PCB que es una estructura del tipo `struct task_struct` y una estructura del tipo `struct thread_info`.
 - En Linux, el PCB es la estructura `struct task_struct`. En esta estructura aparece todo tipo de información sobre un proceso.
 - La estructura `struct thread_info` contiene información de bajo nivel sobre el proceso

```
1 [|||||] 24.6% Tasks: 4, 1 thr; 1 running
2 [|||||] 28.0% Load average: 0.52 0.58 0.59
3 [|||||] 25.5% Uptime: 00:04:04
4 [|||||] 43.0% Time: 16:47:17
Mem[|||||] 12.2G/63.4G Hostname: EliteDesk
Swp[|||||] 0K/55.8G Avg[|||||] 30.4%

PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
1 root 20 0 8936 312 268 S 0.0 0.0 0:00.10 /init
11 root 20 0 8936 220 176 S 0.0 0.0 0:00.00 /init
12 oaromerov 20 0 18080 3600 3496 S 0.0 0.0 0:00.10 -bash
30 oaromerov 20 0 16188 2292 1524 R 0.0 0.0 0:00.00 htop
10 root 20 0 8936 312 268 S 0.0 0.0 0:00.00 /init
```



Operaciones con procesos

Los sistemas que administran procesos deben ser capaces de realizar ciertas operaciones sobre y con los procesos. Tales operaciones incluyen:

- crear y destruir un proceso
- suspender y reanudar un proceso
- cambiar la prioridad de un proceso
- bloquear y "desbloquear" un proceso
- planificar un proceso (asignarle la CPU)
- permitir que un proceso se comuniquen con otro (a esto se denomina comunicación entre procesos, y se estudiará en el tema de procesos concurrentes).

Crear un proceso implica muchas operaciones, tales como:

- buscarle un identificador
- insertarlo en la tabla de procesos
- determinar la prioridad inicial del proceso
- crear el PCB
- asignar los recursos iniciales al proceso

UNIX	WIN32	Descripción
Fork	CreateProcess	Crea un proceso nuevo
Waitpid	WaitForSingleObject	Puede esperar a que un proceso termine
Execve	(ninguna)	CreateProcess= fork + execve
Exit	ExitProcess	Termina la ejecución
Open	CreateFile	Crea un archivo o abre uno existente
Close	CloseHandle	Cierra un archivo
Read	ReadFile	Lee datos de un archivo
Write	WriteFile	Escribe datos en un archivo
Lseek	SetFilePointer	Mueve el apuntador de archivo
Stat	GetFileAttributesEx	Obtiene diversos atributos de archivo
Mkdir	CreateDirectory	Crea un directorio nuevo
Rmdir	RemoveDirectory	Elimina un directorio vacío
Link	(ninguna)	Win32 no maneja enlaces
Mount	(ninguna)	Win32 no maneja montajes
unmount	(ninguna)	Win32 no maneja montajes
Chdir	SetCurrentDirectory	Cambia el directorio de trabajo actual
Kill	TerminateProcess	Elimina un proceso
Time	GetLocalTime	Obtiene la hora actual

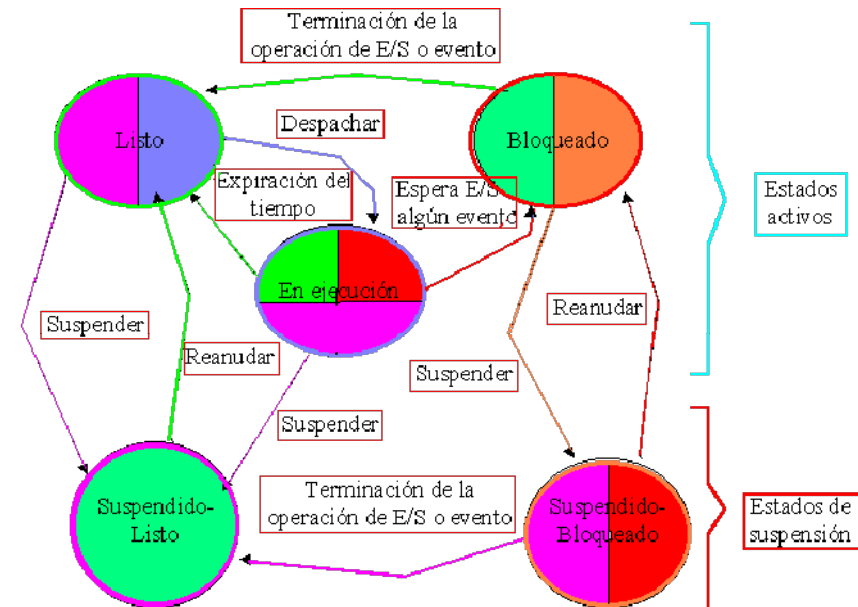
Suspensión y reanudación

Estas operaciones son importantes por diversas razones.

- Si un sistema está funcionando mal, y es probable que falle, se puede suspender los procesos activos para reanudarlos cuando se haya corregido el problema.
- Un usuario que desconfíe de los resultados parciales de un proceso puede suspenderlo (en lugar de abortarlo) hasta que verifique si el proceso funciona correctamente o no.
- Algunos procesos se pueden suspender como respuesta a las fluctuaciones (bajas y altas) a corto plazo de la carga del sistema, y reanudarse cuando las cargas vuelvan a niveles normales.

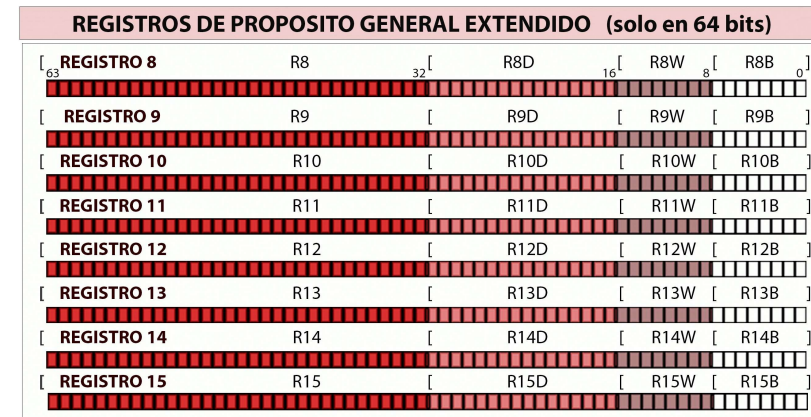
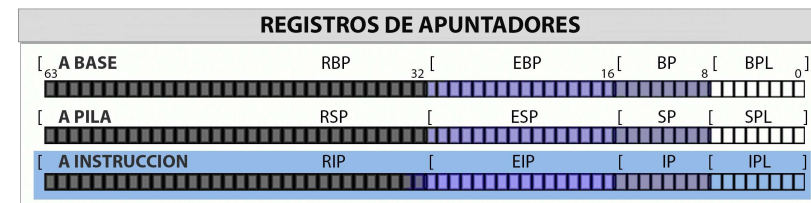
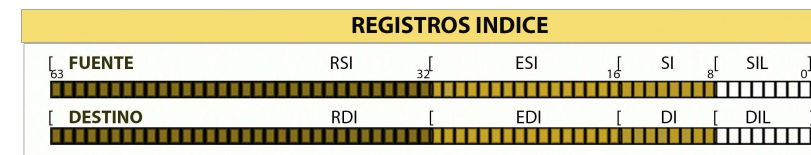
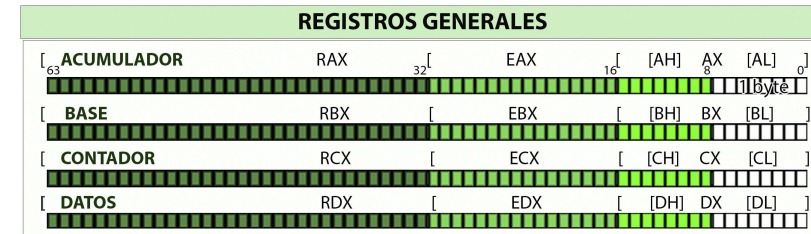
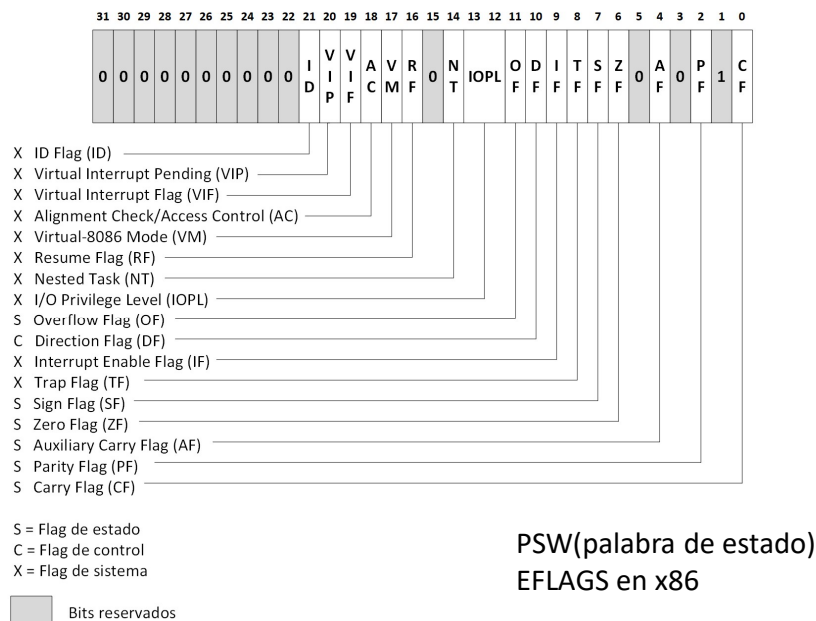
Se han añadido dos estados nuevos: **suspendido_listo** y **suspendido_bloqueado**. El rango de procesos en el azul mantiene los estados activos, y debajo tenemos los estados suspendidos.

Una suspensión puede ser iniciada por el propio proceso o por otro: En un sistema con un procesador, el proceso en ejecución puede suspenderse a sí mismo; ningún otro proceso podría estar en ejecución al mismo tiempo para realizar la suspensión (aunque otro proceso sí podría solicitar la suspensión cuando se ejecute). En un sistema de múltiples procesadores, un proceso en ejecución puede suspender a otro que se esté ejecutando en ese mismo momento en un procesador diferente.



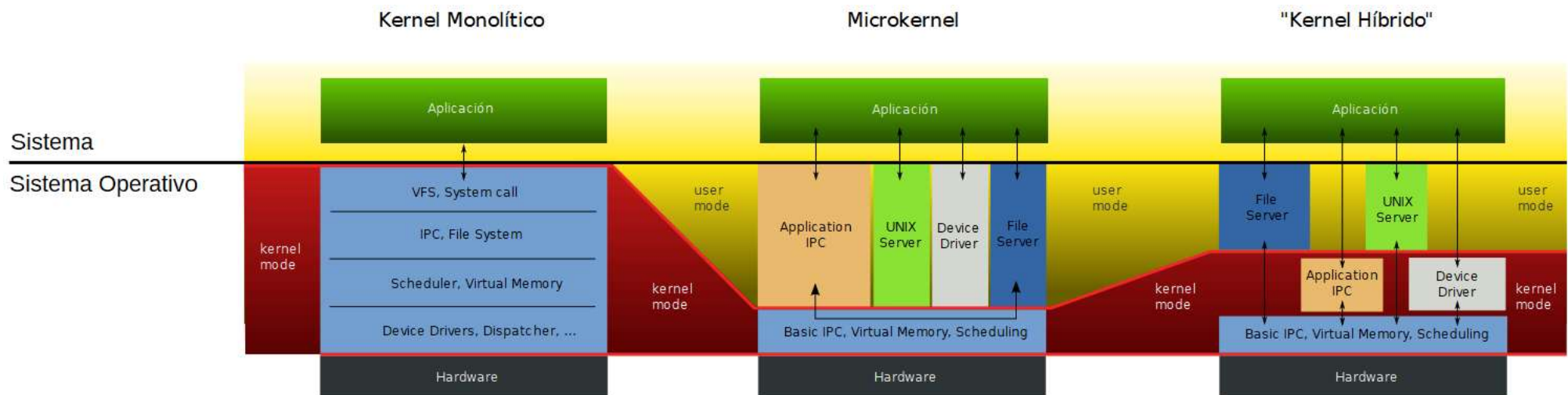
Modos de Ejecución

- Ciertas instrucciones máquina pueden ejecutarse sólo en **modo privilegiado o supervisor**: la lectura o modificación de registros de control (como la palabra de estado), instrucciones primitivas de E/S e instrucciones relativas a la gestión de memoria y solamente se puede acceder a ciertas zonas de memoria en el modo privilegiado.
- El modo de menor privilegio se conoce como **modo usuario**, y es donde se ejecutan la mayoría de los programas.



La razón por la que se usan dos modos

Es necesario proteger al sistema operativo y a las estructuras de datos importantes, tales como los bloques de control de procesos, de las inferencias de los programas de usuario. En el modo núcleo o privilegiado, el software tiene control completo del procesador y de todas las instrucciones, registros y memoria. Hay un bit en la PSW(palabra de estado), que indica el modo de ejecución. El bit se cambia como respuesta a ciertos sucesos tales como una llamada al sistema y, así, se cambia de modo. Sin embargo, surgen diversas cuestiones de diseño debido al interrogante ¿qué sucesos provocan un cambio de procesos?. Se debe hacer una distinción entre **cambio de contexto** y **cambio de proceso**.



Cambio de Proceso

Un cambio de proceso puede suceder en cualquier instante en el que el sistema operativo gana el control de la CPU.

En primer lugar, se van a tener en cuenta las **interrupciones del sistema**, que son de dos clases:

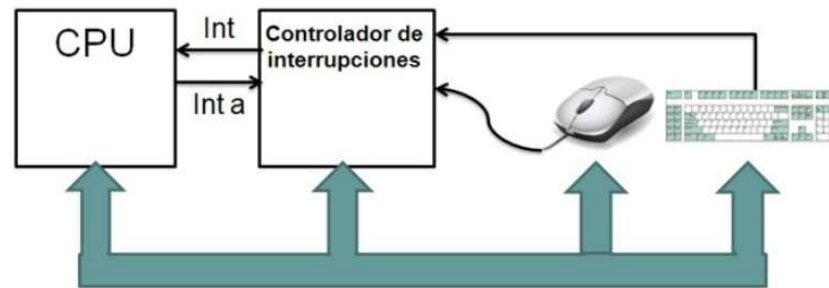
- La originada por algún tipo de suceso que es externo e independiente del proceso que se está ejecutando, como la culminación de una E/S.
- La que tiene que ver con una condición de error o excepción generada dentro del proceso que se está ejecutando, como un intento ilegal de acceso a un fichero, una división entre cero, una instrucción máquina con código de operación no contemplado.

El sistema operativo puede activarse mediante una llamada al sistema desde el programa que se está ejecutando. Por ejemplo, está ejecutándose un proceso de usuario y se llega a una instrucción que solicita una operación de E/S, tal como abrir un fichero. Esta llamada provoca la transferencia a una rutina que forma parte del código del sistema operativo. Por lo general (aunque no siempre) el uso de una llamada al sistema hace que el proceso de usuario pase al estado bloqueado.

Interrupciones del sistema

Cuando un dispositivo acaba una operación de E/S debe de poder comunicárselo al programa que espera su finalización, para que así, el programa pueda proseguir su ejecución.

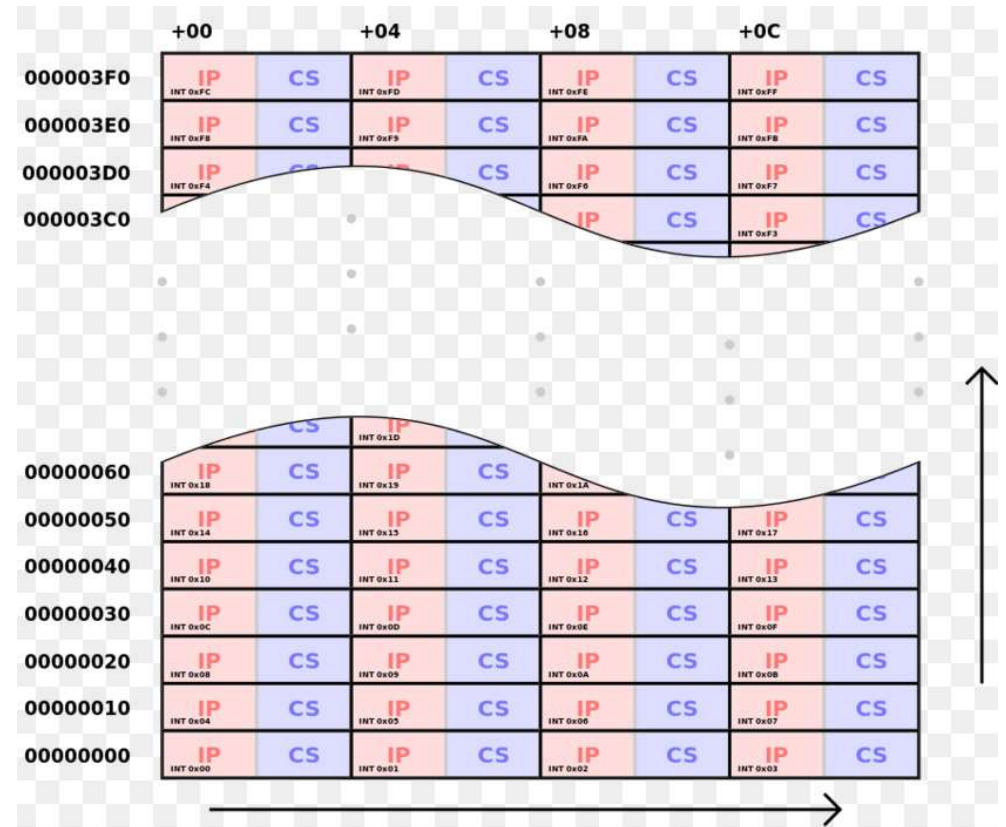
Para indicar el fin de la operación el controlador manda una interrupción a la CPU.



Una interrupción es una señal eléctrica. Provoca que el contador del programa y el registro de PSW del programa en ejecución se salven en un lugar seguro de memoria, para, a continuación, cargar el contador de programa con una **dirección fija de memoria** donde reside un programa del sistema operativo que gestiona la interrupción (se ejecutará un código para indicar al programa que esperaba la finalización de la operación de E/S que ésta ya terminó). Cuando este programa acaba su trabajo, ejecuta una instrucción de retorno de interrupción, la cual restaura el contador de programa y la PSW del programa interrumpido, continuando así su ejecución sin que éste sea consciente de que ha sido interrumpido (E/S controlada por interrupción).

Vector de interrupciones x86

INT (Hex)	IRQ	Common Uses
00-01	Exception Handlers	
02	Non-Maskable IRQ	Non-Maskable IRQ (Parity Errors)
03-07	Exception Handlers	
08	Hardware IRQ0	System Timer
09	Hardware IRQ1	Keyboard
0A	Hardware IRQ2	Redirected (PIC2)
0B	Hardware IRQ3	Serial Comms. COM2/COM4
0C	Hardware IRQ4	Serial Comms. COM1/COM3
0D	Hardware IRQ5	Reserved / SoundCard
0E	Hardware IRQ6	Floppy Disk Controller
0F	Hardware IRQ7	Parallel Comms
10-6F	Software Interrupts	-
70	Hardware IRQ8	Real Time Clock
71	Hardware IRQ9	Redirected IRQ2
72	Hardware IRQ10	Reserved
73	Hardware IRQ11	Reserved
74	Hardware IRQ12	PS/2 Mouse
75	Hardware IRQ13	Math's Co-Processor
76	Hardware IRQ14	Hard Disk Drive
77	Hardware IRQ15	Reserved
78-FF	Software Interrupts	-



- En la arquitectura x86 es indicado por el registro IDTR
- Durante la inicialización se encuentra ubicado en los primeros 1024 bytes de memoria física
- Linux desplaza el vector de interrupciones a otra posición de memoria, reemplazando el del BIOS.

Vector de interrupción en Hex	Descripción
00h	CPU: Ejecutado después de un intento de división por cero o cuando un cociente no cabe en el destino
01h	CPU: Ejecutado después de cada instrucción de código de máquina mientras la bandera de trazado esté activo
02h	CPU: NMI . Usado por el Power On Self Test (POST) para errores de memoria
03h	CPU: La más baja interrupción no reservada, es usada exclusivamente para depuración, y el manejador <code>INT 03h</code> siempre es implementado por un programa de depuración
04h	CPU: Numeric Overflow. Usualmente causado por la instrucción <code>INTO</code> cuando la bandera de desbordamiento está activada
05h	Ejecutado cuando es presionado Shift-Print Screen , así como cuando la instrucción <code>BOUNDh</code> detecta una falla de límite
06h	CPU: Llamada cuando una excepción de opcode indefinido (código inválido). Usualmente instalado por el sistema operativo
07h	CPU: Llamada cuando se intenta usar una instrucción de coma flotante y no está disponible un coprocesador numérico
08h	IRQ0: Implementada por el componente de temporización del sistema; llamado 18,2 veces por segundo (una cada 55 ms) por el PIC
09h	IRQ1: Llamada después de que cada tecla es presionada o soltada (como también durante el tiempo en que una tecla es mantenida presionada)
0Bh	IRQ3: Llamada por los puertos seriales 2 y 4 (COM2 y COM4) cuando es necesaria atención
0Ch	IRQ4: Llamada por los puertos seriales 1 y 3 (COM1 y COM3) cuando es necesaria atención
0Dh	IRQ5: Llamada por el controlador del disco duro (PC/XT) y el segundo puerto paralelo LPT2 (AT) cuando es necesaria atención
0Eh	IRQ6: Llamada por el controlador de disco floppy cuando es necesaria atención
0Fh	IRQ7: Llamada por el primer puerto paralelo LPT1 (impresora) cuando es necesaria atención

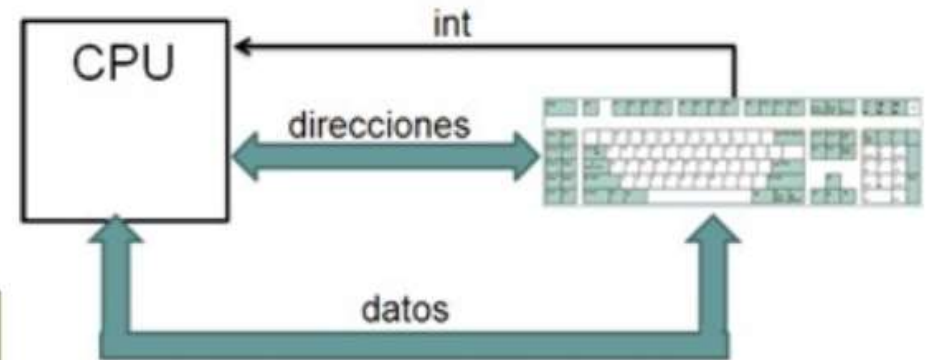
10h	Servicios de video - Instalado por el BIOS o el sistema operativo; llamado por programas de software		
	AH	Descripción (Ing)	Descripción (Esp)
	00h	Set Video Mode	Selecciona modo de vídeo
	01h	Set Cursor Shape	Ajusta forma del cursor
	02h	Set Cursor Position	Ajusta la posición del cursor
	03h	Get Cursor Position And Shape	Lee la posición y la forma del cursor
	04h	Get Light Pen Position	Lee la posición del Light Pen
	05h	Set Display Page	Selecciona la Página de exhibición
	06h	Clear/Scroll Screen Up	Borra la pantalla / Desplazamiento (Scroll) hacia arriba
	07h	Clear/Scroll Screen Down	Borra la pantalla / Desplazamiento (Scroll) hacia abajo
	08h	Read Character and Attribute at Cursor	Lee un carácter y su atributo en la posición del cursor
	09h	Write Character and Attribute at Cursor	Escribe un carácter y su atributo en la posición del cursor
	0Ah	Write Character at Cursor	Escribe un carácter en la posición del cursor
	0Bh	Set Border Color	Ajusta el color del borde de la pantalla
	0Eh	Write Character in TTY Mode	Escribe un carácter en modo TTY
	0Fh	Get Video Mode	Lee el modo de vídeo
	13h	Write String	Escribe un string

```
MOV AH, 0Eh ; Imprime carácter en la pantalla
MOV AL, '!' ; carácter a imprimir
INT 10h ; Llamada a las funciones de video del BIOS
```

Secuencia:

- Teclado interrumpe
- CPU termina instrucción
- CPU pide datos al teclado
- Teclado entrega datos
- Sigue pidiendo hasta que no queden más datos
- CPU le dice que ya no siguen conversando
 - EOI
- CPU vuelve a lo que estaba haciendo antes

I
S
R



Interrupt Service Routine

Clasificación de los tipos de interrupción (pueden ser criterios distintos):

Un usuario realiza una llamada al sistema, concretamente una instrucción trampa (a veces, a las llamadas al sistema se les llaman interrupciones software).

Cuando en un proceso de usuario se da una condición de error (división entre cero, index out of range, page fault, etc.). Esta condición puede tratarse como una "interrupción interna" generada por el procesador y ser gestionada en primera instancia por una rutina de interrupción. A estas condiciones de error algunas veces se les llama excepciones.

Se intenta ejecutar una instrucción reservada estando en modo usuario.

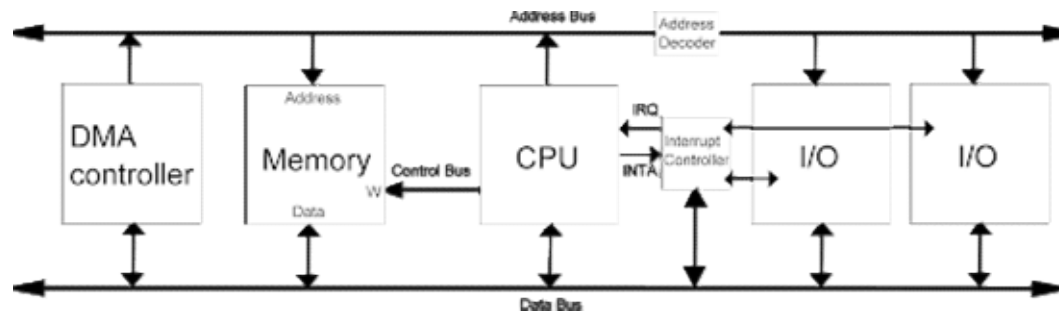
Un controlador de un dispositivo de E/S genera una interrupción para indicar el fin de la E/S.

La ocurrencia de cualquiera de estas circunstancias, incluida las interrupciones, provoca el cambio automático del procesador de modo usuario a modo supervisor.

El controlador de interrupciones de primer nivel FLIH (First-Level Interrupt Handler) es la parte del sistema operativo responsable de proporcionar la respuesta adecuada a las señales procedentes tanto del exterior de la CPU (interrupciones de un controlador) como de dentro del procesador (excepciones y llamadas al sistema). Este determina el origen de las interrupciones y de iniciar el servicio de las mismas.

DMA y canal.

- Cuando un dispositivo realiza una operación de E/S, por ejemplo, una lectura de una USB, la información leída pasa al buffer del controlador. Después, el programa que inició la lectura ejecuta ciertas instrucciones para copiar esta información desde el buffer hacia la memoria principal.
- La copia se realiza mediante un ciclo, copiando en cada iteración del ciclo un byte o una palabra desde el buffer del controlador a la memoria principal.
- En un controlador que disponga de **DMA** (acrónimo de Direct Memory Access, acceso directo a memoria) la copia del buffer a memoria la realiza el propio controlador; para ello, el programa ha de indicarle al controlador la dirección de memoria de inicio de la copia y el número de bytes a copiar, esto lo hace en el momento de darle la orden de E/S, metiendo esta información en algunos registros del controlador.
- Un **canal** es un pequeño procesador de E/S (es decir, un ordenador que sólo entiende instrucciones de E/S), su utilidad es proporcionar DMA a varios dispositivos, resultando más económico que tener un controlador DMA por dispositivo.



Cambio de Proceso – Por interrupción ordinaria

En una interrupción ordinaria, el control se transfiere primero al gestor de interrupciones, quien lleva a cabo algunas tareas básicas y, después, se salta a la rutina del sistema operativo que se ocupa del tipo de interrupción que se ha producido. Algunos ejemplos de estas interrupciones son:

- **Interrupción de reloj:** Un reloj es un dispositivo que genera interrupciones periódicamente. Ante una interrupción de este tipo, el sistema operativo determina si el proceso en ejecución ha alcanzado el máximo tiempo de ejecución que se le concedió. Si es así, el proceso pasará a estado listo y se asignará la CPU a otro proceso (hay diversos algoritmos para escoger que proceso continúa y se denomina ***política de planificación***).
- **Interrupción de E/S:** El sistema operativo determina exactamente qué acción de E/S ha ocurrido. Si se trata de un evento o suceso por el que esperaban uno o más procesos, entonces el sistema operativo traslada todos los procesos bloqueados en dicho evento al estado listo, y determina (dependiendo de la ***política de planificación***) si reanuda la ejecución del proceso interrumpido o pasa a otro de mayor prioridad.
- **Falta de memoria:** Un proceso hace una referencia a una dirección que no se encuentra en memoria y que debe traerse de memoria secundaria (tema de gestión de la memoria). Después de hacer la solicitud de E/S para traer esa o esas direcciones de memoria, el sistema operativo lleva a cabo un cambio de contexto (***política de planificación***) para reanudar la ejecución de otro proceso; el proceso que cometió la falta de memoria se pasa al estado bloqueado. Después de que las direcciones aludidas se carguen en memoria, dicho proceso se pondrá en estado listo.

Cambio de Proceso – Por una condición de error

En una interrupción del segundo tipo, el sistema operativo determina la gravedad del error:

- Si el error es fatal, el proceso que se estaba ejecutando es eliminado y se produce un cambio de proceso.
- Si no es fatal, la acción del sistema operativo dependerá de la naturaleza del error y del diseño del sistema operativo. Se puede hacer un cambio de proceso o, simplemente, reanudar el mismo proceso que se estaba ejecutando.

Si existe una interrupción pendiente:

- Salvar el contexto del programa en ejecución.
- Poner en el PC la dirección del programa de tratamiento de la interrupción (hacer unas pocas tareas básicas)

Cambio de contexto

- **¿Qué es lo que constituye el contexto?:** Información necesaria para reanudar un programa interrumpido
- **¿Qué información?:** Una parte del PCB que tiene la información de estado del procesador, como el contador de programa y otros registros del procesador, así como la información de la pila.
- **¿Se tiene que hacer algo más?:** Dependerá de lo que ocurra a continuación. La rutina de tratamiento de la interrupción es normalmente un programa corto que lleva a cabo unas tareas básicas relacionadas con la interrupción. Por ejemplo, enviar un acuse de recibo a la entidad que produjo la interrupción (como un módulo de E/S)
- **¿Hay que hacer algo más?:** Depende de si la interrupción va a venir seguida de un cambio de proceso o no, ya que la ocurrencia de una interrupción no siempre causa el cambio de proceso. Es posible que después de que el gestor de interrupciones se haya ejecutado, el proceso que estaba ejecutándose reanude su ejecución. En tal caso, tan sólo hay que guardar la información de estado del procesador y restaurarla para que pueda reanudarse correctamente el proceso interrumpido (estas funciones son realizadas en hardware).

Cambio de contexto

El cambio de contexto es un concepto distinto al cambio de un proceso. Puede ocurrir un cambio de contexto sin cambiar del proceso que está actualmente en estado de ejecución. En tal caso, salvar el contexto y restaurarlo posteriormente involucra un pequeño coste extra. Sin embargo, si el proceso que estaba ejecutándose tiene que pasar a otro estado (listo o bloqueado), el sistema operativo tiene que llevar a cabo cambios substanciales en su entorno (contexto). Los pasos involucrados en un cambio completo de proceso son los siguientes:

1. Salvar el contexto del procesador, incluyendo el contador de programa y otros registros.
2. Actualizar el PCB que estaba en estado de ejecución. Esto implica cambiar el estado del proceso a alguno de los otros estados (listo, bloqueado, suspendido_listo). También se tienen que actualizar otros campos, como uno en el que se guarde la razón por la que se abandona el estado de ejecución y otros con información de contabilidad.
3. Mover el PCB a la cola apropiada (listos, bloqueados por el suceso i, suspendido_listo).
4. Seleccionar otro proceso para ejecución (como veremos en el tema de Planificación de Procesos).
5. Actualizar el PCB seleccionado. Cambiar, por ejemplo, su estado a 'en ejecución'.
6. Actualizar las estructuras de datos de gestión de la memoria. Esto puede hacer falta dependiendo de cómo se gestione la traducción de direcciones (lo dejaremos para los temas sobre memoria). Restaurar el contexto del procesador a aquél que existía en el momento en el que el proceso seleccionado dejó por última vez el estado de en ejecución, cargando los valores previos del contador de programa y de otros registros.

El cambio de proceso, que implica un cambio de contexto, requiere un esfuerzo considerablemente superior al de un cambio de contexto.

Procesos y Threads

El concepto de proceso es más complejo, ya que engloba dos conceptos separados y potencialmente independientes: uno relativo a la propiedad de recursos y otro que hace referencia a la ejecución.

- **Unidad que posee recursos:** A un proceso se le asigna un espacio de memoria y gradualmente se le puede asignar otros recursos como dispositivos de E/S.
- **Unidad a la que se le asigna el procesador:** Un proceso es un flujo de ejecución (una traza) a través de uno o más programas.

En la mayoría de los sistemas operativos, estas dos características son la esencia de un proceso. Sin embargo, son independientes, y pueden ser tratadas como tales por el sistema operativo. Esta distinción ha conducido en los sistemas operativos actuales a desarrollar la construcción conocida como **thread**, cuyas traducciones más frecuentes son hilo, hebra y proceso ligero. Si se tiene esta división de características, la unidad de asignación de la CPU se conoce como hilo, mientras que a la unidad que posee recursos se le llama proceso.

Dentro de un proceso puede haber uno o más hilos de control cada uno con:

- Un estado de ejecución (en ejecución, listo, bloqueado).
- Un contexto de procesador, que se salva cuando no esté ejecutándose.
- Una pila de ejecución.
- Algún almacenamiento estático para variables locales.
- Acceso a la memoria y a los recursos de ese trabajo que comparte con los otros hilos.

Procesos y Threads

Los beneficios clave de los hilos se derivan de las implicaciones del rendimiento:

- Se tarda menos tiempo en crear un nuevo hilo de un proceso que ya existe, en terminarlo, y en hacer un cambio de contexto entre hilos de un mismo proceso.
- Al someter a un mismo proceso a varios flujos de ejecución se mantiene una única copia en memoria del código, y no varias.

Un ejemplo de aplicación que podría hacer uso de los hilos es un servidor WEB. Cada vez que llega una solicitud de un recurso web, se puede generar un nuevo hilo para su gestión. El servidor gestiona multitud de solicitudes, por tanto, se pueden crear y destruir muchos hilos en poco tiempo para dar servicio a estas peticiones. Si el servidor es un multiprocesador, se pueden ejecutar varios hilos de un mismo proceso simultáneamente y en diferentes procesadores.

Planificación de procesos

Planificación

La planificación o *scheduling* es el conjunto de políticas y mecanismos contruidos dentro del sistema operativo que gobiernan la forma de conseguir que los procesos a ejecutar lleguen a ejecutarse.

Tiene como objetivos, además de ofrecer una alta tasa de transferencia (throughput) y una baja latencia (latency), es deseable que un planificador implemente las siguientes características :

- **Equidad:** Todos los procesos tengan las mismas oportunidades de ejecución.
- **Repetitividad:** con cargas de trabajo similares (cantidad de procesos a atender), el planificador debe comportarse de manera similar.
- **Predecibilidad:** el tiempo de terminación de un proceso para cierta carga de trabajo, que debe ser similar para cargas de trabajo parecidas.
- **Maximizar la productividad:** evitar conmutaciones innecesarias para aumentar el rendimiento y reducir la penalización asociada y que el procesador esté la mayoría de tiempo ocupado.
- **Eficiencia:** debe tomar decisiones rápidas para aumentar el rendimiento.
- **Atención de prioridades:** uso de criterios de selección basados en prioridades
- **Degradación uniforme del rendimiento:** a mayores cargas de trabajo el rendimiento debe degradarse uniformemente.
- **Capacidad de respuesta instantánea:** los tiempos de espera para atender a un proceso deben ser aceptables de cara al usuario, generalmente <100ms para seres humanos, minimizando el tiempo de espera.

Momentos en los que se decide la planificación de un proceso

- Cuando un proceso se bloquea en espera de un evento (generalmente al realizar una llamada al sistema para una operación de E/S)
- Cuando se produce una interrupción
 - Interrupción de reloj
 - Interrupción de fin de E/S
- Cuando llega un nuevo proceso
- Cuando finaliza un proceso

Midiendo la respuesta

- Resulta intuitivo que al utilizar un sistema, existe un patrón de uso establecido por las tareas que realizan los usuarios.
- Para cada patron de uso se debe seguir políticas de planificación distintas.

Por ejemplo, en el caso de un proceso interactivo:

- se buscará ubicar al proceso en una cola preferente:
 - para obtener un tiempo de respuesta más ágil
 - para mejorar la percepción del Usuario
- pero en caso de sufrir demoras, es preferible:
 - buscar dar una respuesta consistente, aún si la respuesta promedio es más lenta.

Esto es, si a todas las operaciones sigue una demora de un segundo, el usuario sentirá menos falta de control si en promedio tardan medio segundo, pero ocasionalmente hay picos de cinco.

Midiendo la respuesta – Ticks

En vez de emplear unidades temporales formales (por ejemplo, fracciones de segundo), es común emplear ticks y quants. La razón es que:

- si bien en la computación las velocidades de acceso y uso efectivo cambian constantemente, los conceptos y las definiciones permanecen.
- ambos son parámetros ajustables, una misma implementación puede sobrevivir ajustándose a la evolución del hardware.

Tick: Es una fracción de tiempo durante la cual se puede realizar trabajo útil, esto es, usar el CPU sin interrupción (ignorando las interrupciones causadas por los dispositivos de entrada y salida y otras señales que llegan al CPU).

El tiempo correspondiente a un tick está determinado por una interrupción periódica, emitida por el temporizador (timer). La frecuencia con que ocurre esta señal se establece al inicio del sistema. Por ejemplo, una frecuencia de temporizador de 100 Hertz implica que éste emitirá una señal cada 10 milisegundos.

En Linux (desde versión 2.6.8), un tick dura un milisegundo, en Windows, entre 10 y 15 milisegundos.

Midiendo la respuesta – Quantum

El **Quantum** o **Cuanto** es el tiempo mínimo que un proceso puede hacer uso del procesador.

- Puede ser fijo o variable y puede tener el mismo valor para todos los procesos o distinto.
- Generalmente se encuentra en el rango comprendido entre 10 y 100 milisegundos.
- El quantum de un proceso equivalente a un numero fijo de pulsos o ciclos de reloj. Al ocurrir una interrupción de reloj que coincide con la agitación del quantum se llama al **despachador**, el cual le cede el control de la CPU al proceso seleccionado por planificador.

En **Windows**, dependiendo de la clase de proceso que se trate, un quantum durará entre 2 y 12 ticks (esto es, entre 20 y 180 ms). Se configura en el Registro en Windows al cambiar la prioridad en:

HKEY_LOCAL_MACHINE / SYSTEM / CurrentControlSet / Control / PriorityControl / Win32PrioritySeparation

- 0 - Aplicaciones en primer plano y en segundo plano igualmente responsivas
- 1 - Aplicación en primer plano más responsiva que el fondo
- 2 - Mejor tiempo de respuesta de la aplicación en primer plano

En **Linux**, entre 10 y 200 ticks (10 y 200 milisegundos respectivamente).

Tamaño del Quantum o Cuanto

La determinación del tamaño del cuanto es **vital para la operación efectiva de un sistema** de cómputo: ¿Debe el cuanto ser pequeño o grande?, ¿fijo o variable?, ¿el mismo para todos los usuarios o debe determinarse por separado para cada uno?

- **Si es muy grande**, cada proceso tendrá el tiempo necesario para terminar, de manera que el esquema de planificación por turno rotatorio degenera en uno FIFO.
- **Si es muy pequeño**, el cambio de proceso se convierte en el factor dominante y el rendimiento del sistema se degradará hasta el punto en que la mayor parte del tiempo se invierte en la conmutación del procesador, con muy poco tiempo para ejecutar los programas de los usuarios.

Debe ser lo bastante grande como para que la mayoría de las peticiones interactivas requieran menos tiempo que la duración del cuanto.

Tamaño del Quantum o Cuanto

Por ejemplo,

Supongamos que el cambio de proceso tarda 5 ms, y el cuanto es de 20 ms. Con estos parámetros, se utiliza un mínimo del 20% del tiempo de la CPU en la ejecución del sistema operativo. Para incrementar el uso de la CPU por parte de los procesos de usuario, si se establece un cuanto de 500 ms, el tiempo desperdiciado con este parámetro sería del 1%. Pero si 10 usuarios interactivos oprimieran la tecla ENTER casi al mismo tiempo, 10 procesos se colocarían en la lista de procesos listos. Si la CPU está inactiva, el primero de los procesos comenzaría de inmediato, el segundo comenzaría medio segundo después, etc. Partiendo de que todos los procesos agoten su cuanto, el último proceso deberá esperar 4,5 segundos para poder ejecutarse. Esperar esto para la ejecución de una orden sencilla como *pwd* parece excesivo.

CONCLUSIÓN: un cuanto pequeño disminuye el rendimiento de la CPU, mientras que un cuanto muy largo empobrece los tiempos de respuesta y degenera en el algoritmo FIFO. La solución es adoptar un término medio como 100 ms.

Evaluación del criterio de planificación de un proceso

Los siguientes índices que nos permiten evaluar el planificador:

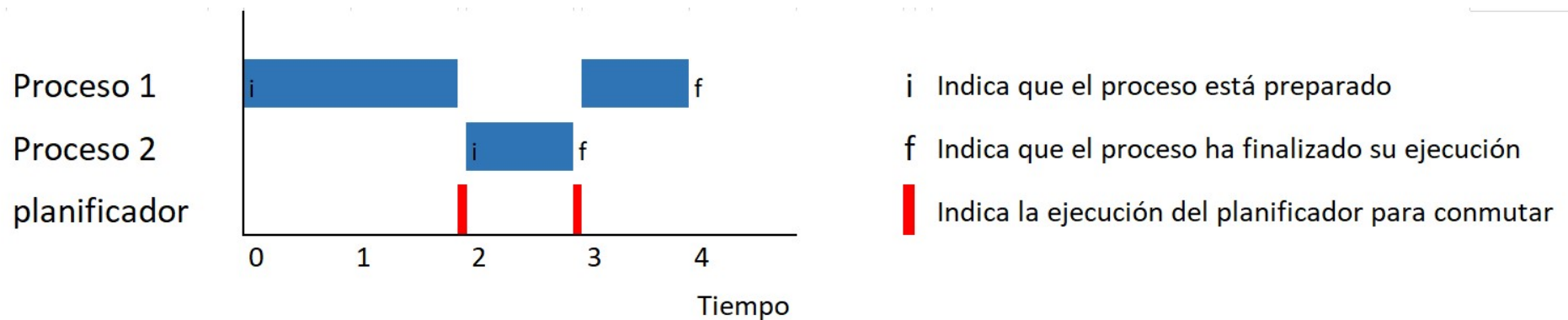
- **Instante de lanzamiento** (I_i): Instante de lanzamiento del proceso.
- **Instante de finalización** (I_f): Instante de finalización del proceso.
- **Tiempo de ejecución** (t): Unidades de tiempo (*ráfagas*) que requiere el proceso para finalizar su ejecución.
- **Tiempo de terminación** ($T = I_f - I_i$): Diferencia entre el instante de lanzamiento y el instante de terminación del proceso. También se conoce como **Tiempo de máquina** o **Tiempo de retorno**.
- **Tiempo perdido** ($T - t$): Es el tiempo durante el cual un proceso no está asignado al procesador, es decir, el tiempo en el que un proceso está en estado preparado (**Tiempo de espera**) o bloqueado (**Tiempo de E/S**).
- **Índice de penalización** ($I_p = T / t$): Indica que un proceso ha tardado $x - 1$ veces más en ejecutarse que si hubiera tenido el procesador en exclusividad y sin que se hubiese producido ningún bloqueo. Nótese que el índice de penalización crece rápidamente para procesos cuyo tiempo de ejecución es pequeño, mientras que, para procesos con tiempo de ejecución mayores, lo hace de manera más suave.

Otros tiempos a tener en cuenta:

- **Tiempo de inactividad**: Es un tiempo específico del procesador. Tiempo durante el cual el procesador está ocioso (*idle*, en inglés), es decir, que no tienen ningún proceso asignado. Un procesador está ocioso cuando no hay procesos en estado preparado.
- **Tiempo del sistema**: Tiempo empleado por el planificador para la conmutación de procesos. Un buen planificador debe tomar muy poco tiempo en tomar la decisión de planificación y realizar la conmutación.
- **Tiempo de espera**: Tiempo desde el instante de lanzamiento hasta que el proceso pasa a estado preparado.

Evaluación del criterio de planificación de un proceso

Ejemplo:



Para el Proceso 1:

$t = 3$: Indica que el proceso 1 requiere 3 unidades de tiempo para ejecutarse.

$T = 4$: unidades de tiempo desde que se lanza hasta que se finaliza el proceso 1.

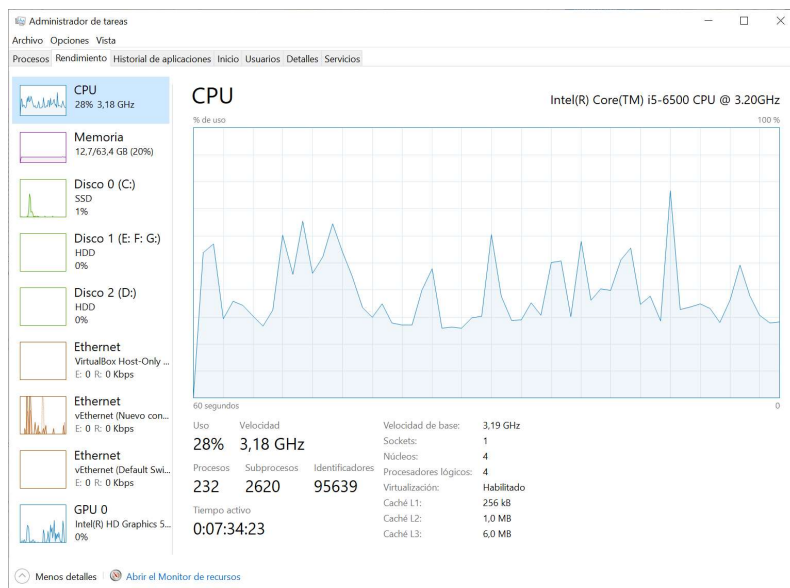
$T - t = 1$: unidades de tiempo durante la cual el proceso 1 no está asignado al procesador, por tanto, no progresa en su actividad.

$I_p = 4/3 = 1.34$: el proceso 1 ha tardado un 34% de tiempo más en ejecutarse que si lo hubiera tenido en exclusividad (al haber tenido que compartir el procesador con el Proceso 2).

En este ejemplo, los tiempos de espera y del sistema se suponen despreciables (en la práctica no lo son, aunque son muy pequeños con respecto a los ilustrados).

Medidas básicas de prestaciones

- **% de utilización de la CPU** (por procesos de usuario): La CPU es un recurso caro que necesita ser explotado, los valores reales suelen estar entre un 40% y un 90%.
- **Rendimiento (throughput)**: Nº de ráfagas (período de tiempo en que un proceso necesita la CPU) por unidad de tiempo; un proceso, durante su vida, alterna ráfagas con bloqueos. Por extensión, también se define como el nº de trabajos por unidad de tiempo.



- Para procesos interactivos, el **Tiempo de respuesta** es el intervalo entre el instante que el usuario da la orden y el instante en que se visualiza la respuesta.
- El **coeficiente de respuesta**, es la razón entre el tiempo de retorno y el tiempo de CPU. Representa una medida de lentitud del computador en responder al usuario, con referencia al tiempo de CPU.

En general, hay que maximizar estos parámetros y minimizar las esperas. Sin embargo, estos objetivos son contradictorios, el dedicar más tiempo de CPU a los usuarios se hace a costa de llamar menos al algoritmo de planificación (menos cambios de proceso), y de simplificarlo. Esto provoca que la CPU se reparta menos equitativamente entre los procesos.

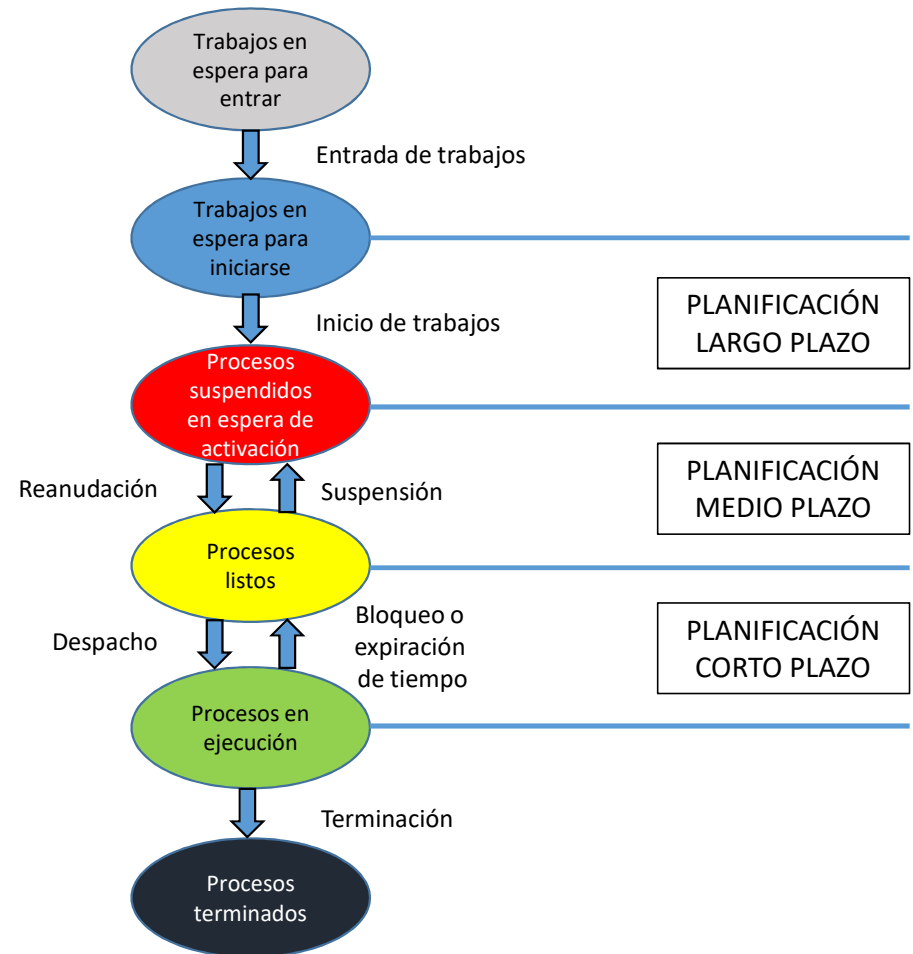
Niveles de Planificación

Está asociado a las cuestiones de:

- Cuándo introducir un nuevo proceso en el sistema.
- Determinar el orden de ejecución de los procesos del sistema.

La planificación está muy relacionada con la gestión de los recursos. Existen tres niveles de planificación (scheduling):

- Planificador a largo plazo.
- Planificador a medio plazo.
- Planificador de la CPU o de corto plazo.



Planificación a largo plazo

- Presente en algunos sistemas que admiten además de procesos interactivos, trabajos por lotes.
- Usualmente, se les asigna una prioridad baja a los trabajos por lotes, utilizándose estos para mantener ocupados a los recursos del sistema durante períodos de baja actividad de los procesos interactivos.
- El objetivo primordial del planificador a largo plazo es el de dar al planificador de la CPU una mezcla equilibrada de trabajos, tales como los limitados por la CPU (utilizan mucho la CPU) o la E/S.
 - Por ejemplo, cuando la utilización de la CPU es baja, el planificador puede admitir más trabajos para aumentar el número de procesos listos y, con ello, la probabilidad de tener algún trabajo útil en espera de que se le asigne la CPU.
 - Cuando la utilización de la CPU llega a ser alta, y el tiempo de respuesta comienza a reflejarlo, el planificador a largo plazo puede optar por reducir la frecuencia de admisión de trabajos.
- Normalmente, se invoca al planificador a largo plazo siempre que un proceso termina.
- La frecuencia de invocación depende de la carga del sistema, pero generalmente es mucho menor que la de los otros dos planificadores. Esta baja frecuencia de uso hace que este planificador pueda permitirse utilizar algoritmos complejos, basados en las estimaciones de los nuevos trabajos.

Planificación a Medio Plazo

En los sistemas de tiempo compartido (por ejemplo, el sistema que controla los cajeros de un banco) hay que atender necesariamente a todos los que se conecten en un momento dado.

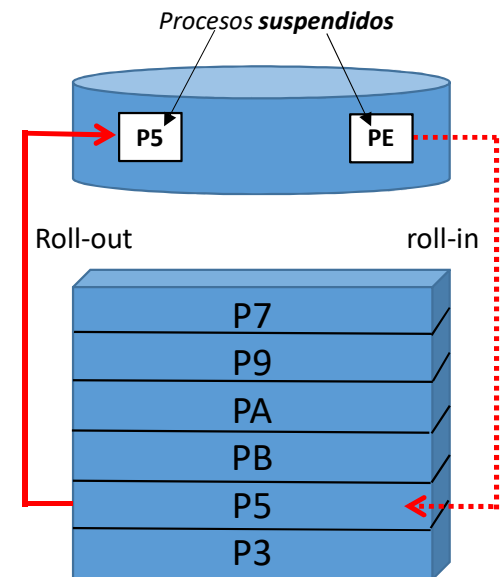
- Varios procesos residen en la memoria principal y debido al tamaño limitado de ésta hace que el número de procesos que residen en ella sea finito, por tanto puede pasar que NO se podrían ejecutar todos.
- Puede ocurrir que todos los procesos en memoria estén bloqueados, desperdiciándose así la CPU.

SOLUCIÓN: Intercambiar procesos enteros (swap) entre memoria principal y memoria secundaria (discos), con esto se aumenta el número de procesos y la probabilidad de una mayor utilización de la CPU.

El planificador a medio plazo es el encargado de regir las transiciones de procesos entre memoria principal y secundaria, intentando maximizar la utilización de los recursos, transfiriendo siempre a memoria secundaria procesos bloqueados, o transfiriendo a memoria principal procesos bloqueados únicamente por no tener memoria. El intercambiador:

- Pasa a disco (“roll-out”), muy rápido con el DMA, procesos bloqueados o listos desde RAM.
- Un proceso pendiente de ejecutar se pasa a RAM (“roll-in”), con lo cual ya puede ejecutarse.

El proceso en disco se dice que está **suspendido**.



Planificador de la CPU o a Corto plazo

- La planificación de la CPU es la conmutación entre procesos. Este despacho es llevado a cabo por un pequeño programa llamado **planificador a corto plazo** o **dispatcher** (despachador).
- Para que el dispatcher conmute el procesador entre dos procesos es necesario realizar un **cambio de proceso**.
- Los acontecimientos que pueden provocar la llamada al dispatcher dependen del sistema (son un subconjunto de las interrupciones), pero algunos son:
 1. El proceso en ejecución acaba su ejecución o no puede seguir ejecutándose (por E/S, operación WAIT, etc.).
 2. Un elemento del sistema operativo ordena el bloqueo del proceso en ejecución.
 3. El proceso en ejecución agota los **quantum** establecidos para hacer uso de la CPU.
 4. Un proceso pasa a estado listo.
- Cuanto menos se llame al dispatcher menos tiempo ocupa la CPU un programa del sistema operativo, y, por tanto, se dedica más tiempo a los procesos del usuario (un cambio de proceso lleva bastante tiempo). Si sólo se activa el dispatcher como consecuencia de los 2 primeros acontecimientos se estará haciendo un buen uso del procesador.

Este criterio es acertado en sistemas por lotes en los que los programas no son interactivos. En un sistema de tiempo compartido no es adecuado, pues un proceso que se dedicara a realizar cálculos, y no realizara E/S, monopolizaría el uso de la CPU. En estos sistemas hay que tener en cuenta el conjunto de todos los procesos, activándose el dispatcher con la circunstancia 3 y, posiblemente, la 4. Los sistemas operativos en que las dos siguientes circunstancias no provocan la activación del dispatcher muestran preferencia por el proceso en ejecución, si no ocurre esto se tiene más en cuenta el conjunto de todos los procesos.

Planificador a Corto plazo – Estrategias de asignación del procesador

- **No apropiativa (*non preemptive*):** El sistema operativo no puede dar el turno a otro proceso, a no ser que el que está en ejecución:
 - Termine
 - Se bloquee (operación de E/S u otro servicio)
 - Ceda voluntariamente el procesador a otro proceso (multiprogramación cooperativa)
 - Ejemplos: MacOS Classic (hasta versión 8), Windows 3.1, RISC OS. Generalmente se usa en sistemas embebidos con limitación de recursos.
- **Apropiativa (*preemptive*):** El sistema operativo puede interrumpir en cualquier momento al proceso activo, pasándolo a LISTO, para dar turno a otro proceso según un **algoritmo de planificación**.
 - Ejemplos: Windows NT, OS/2, Windows 95 y sucesores, MacOS 9, MacOS X, Unix, Linux, ReactOS.

Algoritmos de planificación